

Zadanie 19

Natalia Włodyka

Styczeń 07, 2026

1 Polecenie zadania

Celem zadania jest znalezienie miejsca zerowego funkcji $p(\alpha)$ dla $x = 7$, będącą pochodną wielomianu interpolacyjnego Lagrange'a $l(x, \alpha)$, w prawym końcu przedziału $\alpha \in [-10, 10]$.

2 Wyznaczenie wzoru $p(\alpha)$

Wielomian interpolacyjny można zapisać jako sumę funkcji bazowych Lagrange'a, gdzie x_i, y_i to jego węzły:

$$l(x, \alpha) = \sum_{i=1}^n y_i L_{i(x)}$$

Ponieważ jedynie y_4 zależy od α , zapisuje ten wzór w postaci, gdzie $D(x)$ oznacza sumę składników niezależnych od α :

$$l(x, \alpha) = \alpha L_4(x) + D(x)$$

Możemy z tego wzoru wyznaczyć pochodną w postaci:

$$p(\alpha) = A\alpha + B$$

Gdzie $B = p(0)$, a $A = p(1) - p(0)$.

3 Wyznaczenie miejsca zerowego $p(\alpha)$

Miejsce zerowe wyznaczamy ze wzoru:

$$p(\alpha_0) = A\alpha_0 + B \rightarrow \alpha_0 = -\frac{B}{A} = -\frac{p(0)}{p(1) - p(0)}$$

4 Kod w Pythonie

```

1 import numpy as np
2 from numpy.typing import NDArray
3
4 vector = NDArray[np.float64]
5 numb = np.float64
6
7 def p_alpha(alpha: numb) -> numb:
8     x = np.array([1, 2, 3, 4, 5, 6, 7], dtype=float)
9     y = np.array([1, 0, 1, alpha, 1, 0, 1], dtype=float)
10
11     coeffi = find_coefficients(x, y)
12     return horner(coeffi, 7)
13
14 def lj_x(x: numb, vector_x : vector, j: int) -> numb:
15     n = len(vector_x)
16     numerator: numb = np.float64(1.0)
17     denominator: numb = np.float64(1.0)
18     our_range = [i for i in range(0, n) if i != j]
19
20     for i in our_range:
21         numerator *= x - vector_x[i]
22         denominator *= vector_x[j] - vector_x[i]
23
24     return numerator / denominator
25
26 def find_coefficients(x: vector, f_x: vector) -> vector:
27     n = len(x)
28     lj = np.zeros(n, dtype= numb)
29     for j in range(n):
30         lj[j] = lj_x(0.0, x, j)
31
32     coeffis = np.zeros(n, dtype= numb)
33     f_j = f_x.copy()
34
35     for k in range(n):
36         coeffis[k] = np.sum(lj * f_j)
37
38         for j in range(n):
39             f_j[j] = (f_j[j] - coeffis[k]) / x[j]
40
41     return coeffis
42
43 def horner(coefficients: vector , x: numb) -> numb:
44     result = 0.0
45
46     for i in range(len(coefficients) - 1, 0, -1):
47         result = result * x + i * coefficients[i]
48
49     return result
50
51 def polynomial(x: numb, coeffi: vector) -> numb:
52     lenght = len(coeffi)
53     result: numb = 0.0
54     for i in range(lenght - 1, 0, -1):
55         result += coeffi[i] * x ** i
56
57     return result

```

```

58
59 def function_x(x:vector) -> vector:
60     f_x = np.zeros(len(x), dtype= numb)
61     for i in range(len(x)):
62         f_x[i] = 1/(1 + 5*x[i]*x[i])
63
64     return f_x
65 def main():
66     p_0 = p_alpha(0.0)
67     p_1 = p_alpha(1.0)
68
69     alpha_0 = -p_0/(p_1 - p_0)
70
71     print("Miejsce zerowe p(alpha) wynosi: ", alpha_0, "")
72
73 if __name__ == '__main__':
74     main()

```

5 Miejsce zerowe $p(\alpha)$

Miejsce zerowe $p(\alpha)$ wynosi:

$$\alpha_0 = 2.080000001587457$$